Step 3 — The Writer Synthesizes a Final Report — Solution

> Back to the step · Hint 1 · Hint 2

Story: this adds a Writer — a plain LLM call with no tools attached — that takes both specialists' findings and writes **one** coherent, unified report, not two raw dumps placed side by side. The prompt's wording has to explicitly say *synthesize*, not just hand both texts over and hope. **If not:** the team's final output would read like two separate paragraphs stapled together, and the whole point of having a Writer at all would be lost.

`src/agents/writer_agent.py` (new)

Story: the Writer receives both specialists' findings plus an explicit instruction to synthesize them, and crucially also receives the `web_unavailable` flag so it can honestly name a source that was missing, rather than staying silent about it. **If not:** without that flag actually reaching the prompt, a degraded run would silently produce a report that pretends the web check happened when it never did.

```
# src/agents/writer_agent.py
# The Writer: a plain LLM call that synthesizes the specialists' findings into
# one report, and explicitly notes any source that was unavailable.
from src.utils.logger import get_logger

WRITER_SYSTEM = (
    "You are the Writer. Synthesize the specialists' findings into ONE short, "
    "coherent status report with a title and a few bullet points. Do not just "
    "concatenate the two sources - weave them together. If a source is marked "
    "unavailable, say so plainly in the report; never pretend it was checked."
)

def run_writer(client, task, notes_findings, web_findings, web_unavailable):
    logger = get_logger(__name__)
    web_section = web_findings
    if web_unavailable:
        web_section = "(the web status page could not be reached)"
    user = (
        f"Task: {task}\n\n"
        f"Notes findings:\n{notes_findings or '(none)'}\n\n"
```

```
        f"Web findings:\n{web_section or '(none)'}\n\n"
        "Write the combined report."
    )
    reply = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": WRITER_SYSTEM},
            {"role": "user", "content": user},
        ],
    ).choices[0].message
    logger.info("writer produced the report")
    return reply.content or ""
```

`src/graph.py` — the real writer node

Replace Step 2's placeholder `writer_node` with one that calls `run_writer`:

```
# src/graph.py  (writer node, Step 3)
from src.agents.writer_agent import run_writer
# ...

    def writer_node(state):
        report = run_writer(
            client, state["task"], state["notes_findings"],
            state["web_findings"], state["web_unavailable"],
        )
        return Command(goto=END, update={"report": report})
```

Everything else in `build_graph` stays as it was in Step 2. The Supervisor routes to `writer` once the needed specialists have run (`decide_next` returns `"writer"` when nothing else is pending).

Proving it

Run task 3 (`"Check my notes for Atlas, fetch its status page too, and give one combined report."`). The routing log is `['notes', 'web', 'writer']`, and the report reads as one synthesized status update that draws on **both** sources — the Atlas beta/auth notes and the fetched "82% complete, on track" status — rather than two disconnected paragraphs.

```
Routing: ['notes', 'web', 'writer']

Project Atlas — Status Report
- On track: the status page reports ~82% complete, launch next month.
```

```
- Risks from the notes: auth rewrite is a blocker; a login bug is in review.
- Docs are still pending (notes) and listed as an open risk (status page).
```

The Writer's value is exactly that last bullet — it *connected* a note and the status page into one point, which a concatenation never would. (The full, runnable graph and the integration test that checks the report mentions both sources are in the Step 4 solution.)